

Exception Handling

Trainer’s Name - Ms. Shanmuga Priya

Why we have to handle the exception?

- To provide meaningful error message to the user.
- Exception always happens only during run time.

ArrayIndexOutOfBoundsException

```
int a[]=new int[5];//declaration and instantiation
    a[0]=10;//initialization
    a[1]=20;
    a[2]=70;
    a[3]=40;
    a[4]=50;
    System.out.println(a[6]);
```

NullPointerException

```
String name = null;
System.out.println(name.length());
```

ArithmeticException

```
int a = 5;
int b = 0;
int result = a/b;
System.out.println(result);
```

NumberFormatException

```
Integer.partseInt("234") - valid
Integer.partseInt("eedf") - NumberFormatException
```

When exception happens,
1)java creates one exception object (type is based on the exception)
2)java throws that exception object

When ArithmeticException exception happens,
1)java creates one ArithmeticException object
2)java throws that exception object

How to handle the Exception?

1)Identify the exception causing stmt and put the exception causing stmt in a try block.

2)Followed by the try block add the catch block and catch that exception object. In the catch block provide the meaningful error message to the user.

While adding multiple catch blocks , add higher level of exception class at last.

```
try {
	int i = 1;
	int j = 0;
	System.out.println(i / j);
}

catch (ArithmeticException e) {
	System.out.println("ArithmeticException - Dr can not be zero");
}

catch (ArrayIndexOutOfBoundsException e) {
	System.out.println("ArrayIndexOutOfBoundsException - check the array values");
}

catch (NumberFormatException e) {
	System.out.println("ArrayIndex OutOfBoundsException = String does not contains valid
	number");
}

catch (Exception exception) {
	System.out.println(exception);
}
```

Exception class Hierarchy

class ArithmeticException extends RuntimeException

class RuntimeException extends Exception

public class Exception extends Throwable

Super class of Exception class - Throwable

Super class of Throwable is Object

throw keyword

When ArithmeticException raised what java does?
Creates object for ArithmeticException class and throws that ArithmeticException object.

throw new ArithmeticException("Cannot divide by zero ");
Writing code to create and throw the ArithmeticException object.

Usage – User Defined Exceptionclass

Checked Exception
- IOException
- SQLException
- ClassNotFoundException
- FileNotFoundException
- InterruptedException
- CloneNotSupportedException

Unchecked exceptions

- ArithmeticException
- NullPointerException
- NumberFormatException
- ArrayIndexOutOfBoundsException
- StringIndexOutOfBoundsException
- ClassCastException

InterruptedException

Thread.sleep(1000) - During compile time
error -unreported exception InterruptedException

sleep method throws the InterruptedException

Since the InterruptedException is the checked exception, it is mandatory to add try and catch or use throws to avoid the compile time error.

checked exception :
Compiler forces to handle that exception during compile time.

unchecked exception:
Compiler does not forces to handle that exception during compile time.

Difference between try catch and throws

To handle the exception properly - try catch
To avoid the compile time error unreported exception – throws.

Exception is not properly handled with throws.
So,always handle the exception using try and catch only.

Class not extending the RunTimeException directly or indirectly - Checked exception.
Class is extending the RunTimeException directly or indirectly - Unchecked exception

public class ArithmeticException extends RuntimeException{...} - directly extends the RunTimeException
ArithmeticException - Unchecked exception class

public class ArrayIndexOutOfBoundsException extends IndexOutOfBoundsException
public class IndexOutOfBoundsException extends RuntimeException
public class RuntimeException extends Exception
public class Exception extends Throwable
super class of Throwable is Object

ArrayIndexOutOfBoundsException : indirectly extends RunTimeException

User defined exception class

1) Define the class - Give meaningful name to the class
class name should end with Exception. Example : InvalidAgeException

2) throw new InvalidAgeException("Age must be 18 or above");

Exception propagation:

In the current method, if there is no code to handle the exception, exception is propagated to the caller method, java checks whether that caller method handles that exception.

THANK YOU